1 Model Architecture

The proposed model is a convolutional neural network (CNN) designed to predict a vehicle's future trajectory over a 5-second horizon using multi-modal inputs: two RGB images from a front-facing GoPro camera (each $640 \times 640 \times 3$ pixels, processed simultaneously to capture temporal context from consecutive frames) and three scalar values—speed, acceleration, and turn rate—derived from GPS. The output is a tensor of shape $(10, 2)$, representing 10 two-dimensional vectors $(\Delta x, \Delta y)$ for the vehicle's travel distance in the 2D road plane over 0.5-second intervals. The GPS data enables automatic generation of ground-truth trajectory labels by recording future positions at 0.5-second intervals. To exploit temporal relationships between the two images, cross-processing is introduced via cross-attention mechanisms after intermediate convolutional layers, allowing feature maps from one image to influence the other. The architecture is optimized to approach ∼22 million parameters, leveraging the computational capacity of a GPU capable of training a 22.4-million-parameter model (e.g., YOLOv11m). The architecture is structured as follows:

1. **Image Processing Branch (Convolutional Backbone with Cross-Processing for Two Images)**:
    - **Layer 1**: Convolutional layer with 64 filters, kernel size 3×3, ReLU activation, input shape $(640, 640, 3)$, applied to the first image.
    - **Layer 2**: Batch normalization.
    - **Layer 3**: Max-pooling layer with 2×2 pool size.
    - **Layer 4**: Convolutional layer with 128 filters, kernel size 3×3, ReLU activation.
    - **Layer 5**: Batch normalization.
    - **Layer 6**: Convolutional layer with 64 filters, kernel size 3×3, ReLU activation, input shape $(640, 640, 3)$, applied to the second image (shared weights with Layers 1–5).
    - **Layer 7**: Batch normalization.
    - **Layer 8**: Max-pooling layer with 2×2 pool size.
    - **Layer 9**: Convolutional layer with 128 filters, kernel size 3×3, ReLU activation.
    - **Layer 10**: Batch normalization.
    - **Layer 11**: Cross-attention layer, where feature maps from the first image (from Layer 5, $320 \times 320 \times 128$) attend to feature maps from the second image (from Layer 10, $320 \times 320 \times 128$), and vice versa, producing enhanced feature maps for each image ($320 \times 320 \times 128$).
    - **Layer 12**: Max-pooling layer with 2×2 pool size, applied to the first image's enhanced feature maps.
    - **Layer 13**: Convolutional layer with 256 filters, kernel size 3×3, ReLU activation.
    - **Layer 14**: Batch normalization.
    - **Layer 15**: Convolutional layer with 256 filters, kernel size 3×3, ReLU activation.
    - **Layer 16**: Batch normalization.
    - **Layer 17**: Max-pooling layer with 2×2 pool size, applied to the second image's enhanced feature maps.
    - **Layer 18**: Convolutional layer with 256 filters, kernel size 3×3, ReLU activation.
    - **Layer 19**: Batch normalization.
    - **Layer 20**: Convolutional layer with 256 filters, kernel size 3×3, ReLU activation.
    - **Layer 21**: Batch normalization.
    - **Layer 22**: Cross-attention layer, where feature maps from the first image (from Layer 16, $160 \times 160 \times 256$) attend to feature maps from the second image (from Layer 21, $160 \times 160 \times 256$), and vice versa, producing enhanced feature maps ($160 \times 160 \times 256$).
    - **Layer 23**: Max-pooling layer with 2×2 pool size, applied to the first image's enhanced feature maps.
    - **Layer 24**: Convolutional layer with 512 filters, kernel size 3×3, ReLU activation.
    - **Layer 25**: Batch normalization.

- **Layer 26**: Convolutional layer with 512 filters, kernel size 3×3, ReLU activation.
- **Layer 27**: Batch normalization.
- **Layer 28**: Max-pooling layer with 2×2 pool size, applied to the second image's enhanced feature maps.
- **Layer 29**: Convolutional layer with 512 filters, kernel size 3×3, ReLU activation.
- **Layer 30**: Batch normalization.
- **Layer 31**: Convolutional layer with 512 filters, kernel size 3×3, ReLU activation.
- **Layer 32**: Batch normalization.
- **Layer 33**: Concatenation layer to merge the feature maps from both images (e.g., combining two $40 \times 40 \times 512$ feature maps into $40 \times 40 \times 1024$).
- **Layer 34**: Flatten layer to vectorize the concatenated feature maps (e.g., $40 \times 40 \times 1024 = 1,638,400$ units).

2. **Dynamic Input Processing Branch (Speed, Acceleration, Turn Rate)**:

   - **Layer 35**: Dense layer with 32 units, ReLU activation, input shape (3), processing the concatenated vector of speed (e.g., meters per second), acceleration (e.g., meters per second^2), and turn rate (e.g., radians per second).
   - **Layer 36**: Batch normalization.

3. **Feature Fusion**:

   - **Layer 37**: Concatenation layer to merge the flattened image features (from Layer 34) and the processed dynamic features (from Layer 36), producing a fused feature vector (e.g., $1,638,400 + 32 = 1,638,432$ units).

4. **Fully Connected Layers**:

   - **Layer 38**: Dense layer with 2048 units, ReLU activation, to integrate multi-modal features.
   - **Layer 39**: Dropout layer with 0.5 dropout rate to mitigate overfitting.
   - **Layer 40**: Dense layer with 512 units, ReLU activation.
   - **Layer 41**: Dropout layer with 0.5 dropout rate.
   - **Layer 42**: Dense layer with 128 units, ReLU activation.

5. **Output Layer**:

   - **Layer 43**: Dense layer with 20 units, linear activation, producing a flattened vector of 10 (x, y) travel distances.
   - **Layer 44**: Reshape layer to format the output as a tensor of shape $(10, 2)$, where each vector represents the relative displacement $(\Delta x, \Delta y)$ over a 0.5-second interval.